TCSS 343 – Design and Analysis of Algorithms

Longest Common Subsequence

Sub-Sequences and Sub-Strings

- A sequence is an ordered set.
  - A subsequence is created from a sequence by deleting some elements.
  - A substring is a subsequence that also appears consecutively in the original sequence.

- Example: “BANDANA”
  - “BANANA” is a subsequence but not a substring.
  - “BAND” is a substring.

- Two sequences may have subsequences and substrings in common.

- Example: “BANDANA” and “MONTANA”
  - What is the longest common subsequence?
    - “NANA” of length 4.
  - What is the longest common substring?
    - “ANA” of length 3.

Longest Common Subsequence

Longest Common Subsequence (LCS)
Input: $A[1 \dots n]$ and $B[1 \dots m]$ two sequences of characters
Output: l the length of the longest common subsequence and $C[1 \dots l]$ the common subsequence

Longest Common Substring(LCS2)
Input: $A[1 \dots n]$ and $B[1 \dots m]$ two sequences of characters
Output: l the length of the longest common substring and $C[1 \dots l]$ the common substring

- Subsequence
- Substring
- Longest_common_subsequence_problem
- Longest_common_substring_problem

Dynamic Programming

- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Additional resources on Dynamic Programming:
  - Dynamic_programming
  - Optimal_substructure
  - Overlapping_subproblems

Longest Common Subsequence

- What sub-problems should we consider?
  - We will consider prefixes of the subsequences.
  - Recall a prefix of a string $s_1 s_2 \cdots s_n$ is $s_1 s_2 \cdots s_k$ where $1 \leq k \leq n$.
  - Since we have two input sequences, we will consider finding the longest common subsequences of all pairs of prefixes of the input sequences.
- Let $LCS(n, m)$ express the length of the longest common substring on inputs of length n and m.

- Let's consider two cases:
  - The last character of A and B are the same.
    - Example: $ABBCDA$ and $AAACBA$
    - We can find the LCS of prefixes of A and B of size $n-1$ and $m-1$ respectively. We can then add the last character to this LCS.
  - The last character of A and B are not the same.
    - Example: $ABBCDA$ and $AAACBX$
    - We can now consider two possible sub-problems. We take the prefix of A of length $n-1$ and the complete string B or we take the complete string A with the prefix of B of length $m-1$. One of these sub-problems will be maximal.

$$LCS(n,m) = \begin{cases} 0 & \text{if } n = 0 \text{ or } m = 0 \\ LCS(n-1, m-1) + 1 & \text{if } A[n] = B[m] \\ \max\{LCS(n-1, m), LCS(n, m-1)\} & \text{if } A[n] \neq B[m] \end{cases}$$

Longest Common Subsequence

$$LCS(n,m) = \begin{cases} 0 & \text{if } n = 0 \text{ or } m = 0 \\ LCS(n-1, m-1) + 1 & \text{if } A[n] = B[m] \\ \max\{LCS(n-1,m), LCS(n,m-1)\} & \text{if } A[n] \neq B[m] \end{cases}$$

- Do we have overlapping sub-problems?
  - Notice that $LCS(n-1,m)$ and $LCS(n,m-1)$ both make recursive calls to $LCS(n-1,m-1)$ so we have overlapping sub-problems.
- What data structure should we use to store the solutions to the sub-problems?
  - Since we use two parameters in our self-reduction we will use a two-dimensional array to store the solutions to our sub-problems.
- What order should we solve the sub-problems?
  - We will start at $n = 0$ and $m = 0$.
  - We can go row by row or column by column.

Dynamic Programming Algorithm

$$LCS(n,m) = \begin{cases} 0 & \text{if } n = 0 \text{ or } m = 0 \\ LCS(n-1, m-1) + 1 & \text{if } A[n] = B[m] \\ \max\{LCS(n-1,m), LCS(n,m-1)\} & \text{if } A[n] \neq B[m] \end{cases}$$

```
LCS(A[1 ... n], B[1 ... m])
        Let L[0 ... n][0 ... m] be an array of numbers of size n.
        For i = 0 to n do:
                Let L[i][0] = 0.
        End For
        For j = 0 to m do:
                Let L[0][j] = 0.
        End For
        For i = 1 to n do:
                For j = 1 to m do:
                        If A[i] = B[j] then let L[i][j] = L[i − 1][j − 1] + 1
                        Else let L[i][j] = max{L[i − 1][j], L[i][j − 1]}.
                End For
        End For
        Return L[n][m].
End LCS
```

Dynamic Programming Algorithm

```
LCS(A[1 ... n], B[1 ... m])
        …
        For i = 1 to n do:
                For j = 1 to m do:
                        If A[i] = B[j] then let L[i][j] = L[i − 1][j − 1] + 1
                        Else let L[i][j] = max{L[i − 1][j], L[i][j − 1]}.
                End For
        End For
        Return L[n][m].
End LCS
```

		B	A	N	D	A	N	A
	0	0	0	0	0	0	0	0
M	0	0	0	0	0	0	0	0
O	0	0	0	0	0	0	0	0
N	0	0	0	1	1	1	1	1
T	0	0	0	1	1	1	1	1
A	0	0	1	1	1	2	2	2
N	0	0	1	2	2	2	3	3
A	0	0	1	2	2	3	3	4

		B	A	N	D	A	N	A
M		←	←	←	←	←	←	←
O		←	←	←	←	←	←	←
N		←	←	↖	←	←	↖	←
T		←	←	↑	←	←	←	←
A		←	↖	←	←	↖	←	↖
N		←	↑	↖	←	←	↖	←
A		←	↖	↑	←	↖	←	↖

TCSS 343 – Design and Analysis of Algorithms

Longest Common Substring

Longest Common Substring

Longest Common Subsequence (LCS)
Input: $A[1 \dots n]$ and $B[1 \dots m]$ two sequences of characters
Output: l the length of the longest common subsequence and $C[1 \dots l]$ the common subsequence

Longest Common Substring(LCS2)
Input: $A[1 \dots n]$ and $B[1 \dots m]$ two sequences of characters
Output: l the length of the longest common substring and $C[1 \dots l]$ the common substring

- Subsequence
- Substring
- Longest_common_subsequence_problem
- Longest_common_substring_problem

Longest Common Substring

- Substrings must be consecutive and this complicates the problem.
  - Instead of solving Longest Common Substring directly we will consider finding the length of the common suffix for each pair of prefixes of A and B.

- Let $LCS(n, m)$ express the length of the common suffix on inputs of length n and m.

- Let's consider two cases:
  - The last character of A and B are not the same.
    - In this case the length of the common suffix is 0.
  - The last character of A and B are the same.
    - We can find the LCS of prefixes of A and B of size $n-1$ and $m-1$ respectively. We can then add the last character to this LCS.

$$LCS(n,m) = \begin{cases} 0 & \text{if } n = 0 \text{ or } m = 0 \\ 0 & \text{if } A[n] \neq B[m] \\ LCS(n-1, m-1) + 1 & \text{if } A[n] = B[m] \end{cases}$$

Longest Common Subsequence

$$LCS(n,m) = \begin{cases} 0 & \text{if } n = 0 \text{ or } m = 0 \\ 0 & \text{if } A[n] \neq B[m] \\ LCS(n-1, m-1) + 1 & \text{if } A[n] = B[m] \end{cases}$$

- Do we have overlapping sub-problems?
  - There are no overlapping sub-problems.
  - However, to solve the Longest Common Substring problem we will need to know the solutions to all sub-problems of the Longest Common Suffix problem.
- What data structure should we use to store the solutions to the sub-problems?
  - Since we use two parameters in our self-reduction we will use a two-dimensional array to store the solutions to our sub-problems.
- What order should we solve the sub-problems?
  - We will start at $n = 0$ and $m = 0$.
  - We can go row by row or column by column.

Dynamic Programming Algorithm

$$LCS(n,m) = \begin{cases} 0 & \text{if } n = 0 \text{ or } m = 0 \\ 0 & \text{if } A[n] \neq B[m] \\ LCS(n-1, m-1) + 1 & \text{if } A[n] = B[m] \end{cases}$$

```
LCS(A[1 ... n], B[1 ... m])
        Let L[0 ... n][0 ... m] be an array of numbers of size n.
        For i = 0 to n do:
                Let L[i][0] = 0.
        End For
        For j = 0 to m do:
                Let L[0][j] = 0.
        End For
        For i = 1 to n do:
                For j = 1 to m do:
                        If A[i] = B[j] then let L[i][j] = L[i − 1][j − 1] + 1
                        Else let L[i][j] = 0.
                End For
        End For
        Return max{L[i][j]: 1 ≤ i ≤ n and 1 ≤ j ≤ m}.
End LCS
```

Dynamic Programming Algorithm

```
LCS (A[1 ... n], B[1 ... m])
        …
        For i = 1 to n do:
                For j = 1 to m do:
                        If A[i] = B[j] then let L[i][j] = L[i − 1][j − 1] + 1
                        Else let L[i][j] = 0.
                End For
        End For
        Return max{L[i][j]: 1 ≤ i ≤ n and 1 ≤ j ≤ m}.
End LCS
```

		B	A	N	D	A	N	A
	0	0	0	0	0	0	0	0
M	0	0	0	0	0	0	0	0
O	0	0	0	0	0	0	0	0
N	0	0	0	1	0	0	1	0
T	0	0	0	0	0	0	0	0
A	0	0	1	0	0	1	0	1
N	0	0	0	2	0	0	2	0
A	0	0	1	0	0	1	0	3

TCSS 343 – Design and Analysis of Algorithms

Edit Distance

Edit Distance

- Edit distance is the number of edits needed to transform a string of characters into another string of characters.
- In 1965 Vladimir Levenshtein developed the first characterization of edit distance considering these operations:
  - Insertion: Insert a symbol at some position in the string.
  - Deletion: Delete a symbol at some position in the string.
  - Substitution: Replace a symbol in the string with another symbol.
- The Levenshtein Edit Distance is the most common measurement of similarity between strings.
  - Vladimir_Levenshtein
  - Edit_distance
  - Levenshtein_distance
  - Wagner-Fischer_algorithm
- What is the minimum number of edits to transform “MONTANA” into “BANDANA”?



Dynamic Programming

- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Additional resources on Dynamic Programming:
  - Dynamic_programming
  - Optimal_substructure
  - Overlapping_subproblems

Levenshtein Edit Distance

```
Levenshtein Edit Distance (LED)
Input:    A[1…n] and B[1…m] two sequences of characters
Output:   e the minimum number of edits needed to transform A into B and
          a list L of the edits to be performed
```

- Case One: Either $n = 0$ or $m = 0$.
  - In this case the number of edits is equal to the length of the other string.
- Case Two: $A[n] = B[m]$
  - We do not need to transform this character so we look for the edit distance to transform the prefix of A of length $n-1$ into the prefix of B of length $m-1$.
- Case Three: $A[n] \neq B[m]$
  - Deletion: Delete $A[n]$ and transform the prefix of A of length $n-1$ into B.
  - Insertion: Transform A into the prefix of B of length $m-1$ and then add $B[m]$ to the end.
  - Substitution: Transform the prefix of A of length $n-1$ into the prefix of B of length $m-1$ and then substitute out $A[n]$ for $B[m]$.

$$LED(n,m) = \begin{cases} m & \text{if } n = 0 \\ n & \text{if } m = 0 \\ LED(n-1, m-1) & \text{if } A[n] = B[m] \\ \min\{LED(n-1,m-1), LED(n-1,m), LED(n,m-1)\} + 1 & \text{if } A[n] \neq B[m] \end{cases}$$

Levenshtein Edit Distance

$$LED(n,m) = \begin{cases} m & \text{if } n = 0 \\ n & \text{if } m = 0 \\ LED(n-1, m-1) & \text{if } A[n] = B[m] \\ \min\{LED(n-1,m-1), LED(n-1,m), LED(n,m-1)\} + 1 & \text{if } A[n] \neq B[m] \end{cases}$$

- Levenshtein introduced this recursive definition for the edit distance.

- Following this several researchers independently developed the dynamic programming algorithm to calculate edit distance:
  - Vintsyuk, 1968
  - Needleman and Wunsch, 1970
  - Sankoff, 1972
  - Sellers, 1974
  - Wagner and Fischer, 1974
  - Lowrance and Wagner, 1975

Wagner-Fischer Algorithm

$$LED(n,m) = \begin{cases} m & \text{if } n = 0 \\ n & \text{if } m = 0 \\ LED(n-1,m-1) & \text{if } A[n] = B[m] \\ \min\{LED(n-1,m-1), LED(n-1,m), LED(n,m-1)\} + 1 & \text{if } A[n] \neq B[m] \end{cases}$$

```
Levenshtein(A[1 ... n], B[1 ... m])
        Let L[0 ... n][0 ... m] be a 2d-array of numbers.
        For i = 0 to n do:
                Let L[i, 0] = i.
        End For
        For j = 0 to m do:
                Let L[0, j] = j.
        End For
        …
        Return L[n, m].
End Levenshtein
```

Wagner-Fischer Algorithm

$$LED(n,m) = \begin{cases} m & \text{if } n = 0 \\ n & \text{if } m = 0 \\ LED(n-1,m-1) & \text{if } A[n] = B[m] \\ \min\{LED(n-1,m-1), LED(n-1,m), LED(n,m-1)\} + 1 & \text{if } A[n] \neq B[m] \end{cases}$$

```
Levenshtein(A[1 ... n], B[1 ... m])
        Let L[0 ... n][0 ... m] be a 2d-array of numbers.
        …
        For i = 1 to n do:
                For j = 1 to m do:
                        If A[i] = B[j] let L[i, j] = L[i − 1, j − 1].
                        Else Let L[i, j] = min{L[i − 1, j − 1], L[i − 1, j], L[i, j − 1]} + 1.
                End For
        End For
        Return L[n, m].
End Levenshtein
```

Edit Distance

```
Levenshtein(A[1 ... n], B[1 ... m])
        Let L[0 ... n][0 ... m] be a 2d-array of numbers.
        …
        For i = 1 to n do:
                For j = 1 to m do:
                        If A[i] = B[j] let L[i, j] = L[i − 1, j − 1].
                        Else Let L[i, j] = min{L[i − 1, j − 1], L[i − 1, j], L[i, j − 1]} + 1.
                End For
        End For
        Return L[n, m].
End Levenshtein
```

		B	A	N	D	A	N	A
	0	1	2	3	4	5	6	7
M	1	1	2	3	4	5	6	7
O	2	2	2	3	4	5	6	7
N	3	3	3	2	3	4	5	6
T	4	4	4	3	3	4	5	6
A	5	5	4	4	4	3	4	5
N	6	6	5	4	5	4	3	4
A	7	7	6	5	5	5	4	3

TCSS 343 – Design and Analysis of Algorithms

Subset-Sum and Partition

Dynamic Programming

- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Additional resources on Dynamic Programming:
  - Dynamic_programming
  - Subset_sum_problem
  - Partition_problem
  - Knapsack_problem

Subset Sum

- Subset Sum Problem
  - You have a set of positive integers and a target sum and want to know if there is a subset of the set that add up to the target.
  - Example: $\{5,7,2,3,9\}$ and 13?

- Can we express this problem formally?

Subset Sum(SUM)
Input: $S[1 \dots n]$ a list of n positive integers and T a target integer.
Output: $A \subseteq \{1, \dots, n\}$ a set of indexes into S such that $T = \sum_{i \in A} S[i]$ if such an A exists and false otherwise.

- Can we design a brute force algorithm for this problem?
  - We can check every subset to see its sum. How many subsets are there?

$$2^n$$

  - How many additions does it take to check each sum in the worst case?

$$n$$

  - What is the worst-case number of operations carried out by the brute force algorithm?

$$\theta(n2^n)$$

Subset Sum

Subset Sum(SUM)
Input: $S[1 \dots n]$ a list of n positive integers and T a target integer.
Output: $A \subseteq \{1, \dots, n\}$ a set of indexes into S such that $T = \sum_{i \in A} S[i]$ if such an A exists and false otherwise.

- What is a good set of sub-problems to look at?
  - We can reduce the set of numbers to a smaller set.
  - We can also select different target values.
  - For this problem we will consider sub-problems that restrict both dimensions.

- Let $SS(n, T)$ express the output for the Subset Sum problem on input $S[1 \dots n]$ and T. Can we state a self-reduction from this problem to its sub-problems? We consider here only deciding if the subset exists.

$$SS(n,t) = \begin{cases} \text{true} & \text{if } T = 0 \\ \text{false} & \text{if } T > 0 \text{ and } n = 0 \\ SS(n-1,T) \text{ OR } SS(n-1, T - S[n]) & \text{if } T > 0 \text{ and } n > 0 \end{cases}$$

- Do we have overlapping sub-problems?

Data Structure

$$SS(n,t) = \begin{cases} \text{true} & \text{if } T = 0 \\ \text{false} & \text{if } T > 0 \text{ and } n = 0 \\ SS(n-1,T) \text{ OR } SS(n-1,T-S[n]) & \text{if } T > 0 \text{ and } n > 0 \end{cases}$$

- What type of data structure would be appropriate for this problem?
  - In this problem we use two variables to indicate the sub-problem (n and T).
  - So we have a 2-dimensional problem.
  - We will create a 2d array $A[0 \ldots n][0 \ldots T]$ to store the solutions to the sub-problems .

- What order should we evaluate the problems?
  - We will fill in the array considering one item at a time in order from 0 to n.
  - For each item we will consider each target value at a time in order from 0 to T.

Dynamic Programming Algorithm

$$SS(n,t) = \begin{cases} \text{true} & \text{if } T = 0 \\ \text{false} & \text{if } T > 0 \text{ and } n = 0 \\ SS(n-1,T) \text{ OR } SS(n-1,T-S[n]) & \text{if } T > 0 \text{ and } n > 0 \end{cases}$$

```
SubsetSum(S[1 ... n], T)
        Let A[0 ... n][0 ... T] be an array of integers of size n + 1 × T + 1.
        For i = 0 to n:
                Let A[i][0] = true.
        End For
        For j = 1 to T:
                Let A[0][j] = false.
        End For
        For i = 1 to n do:
                For j = 1 to T:
                        Let A[i][j] = A[i − 1][j] ∨ A[i − 1][j − S[i]].
                End For
        End For
        Return A[n][T].
End SubsetSum
```

Dynamic Programming Algorithm

```
SubsetSum(S[1 ... n], T)
        Let A[0 ... n][0 ... T] be an array of integers of size n + 1 × T + 1.
        For i = 0 to n:
                Let A[i][0] = true.
        End For
        For j = 1 to T:
                Let A[0][j] = false.
        End For
        For i = 1 to n do:
                For j = 1 to T:
                        Let A[i][j] = A[i − 1][j] ∨ A[i − 1][j − S[i]].
                End For
        End For
        Return A[n][T].
End SubsetSum
```

$S = \{5,7,2,3,9\}$ $T = 13$

A

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13
	T	F	F	F	F	F	F	F	F	F	F	F	F	F
5	T	F	F	F	F	T	F	F	F	F	F	F	F	F
7	T	F	F	F	F	T	F	T	F	F	F	F	T	F
2	T	F	T	F	F	T	F	T	F	T	F	F	T	F
3	T	F	T	T	F	T	F	T	T	T	T	F	T	F
9	T	F	T	T	F	T	F	T	T	T	T	T	T	F

Exercise: $S = \{1,4,5,3,7\}$ $T = 16$

Design code to recover the subset.

Consider related problems:

Count the number of subsets that sum to T.
Find the subset that sums to T with maximum size.
Find the subset that sums to T with minimum size.

Dynamic Programming Algorithm

```
SubsetSum(S[1 ... n], T)
        Let A[0 ... n][0 ... T] be an array of integers of size n + 1 × T + 1.
        For i = 0 to n:
                Let A[i][0] = true.
        End For
        For j = 1 to T:
                Let A[0][j] = false.
        End For
        For i = 1 to n do:
                For j = 1 to T:
                        Let A[i][j] = A[i − 1][j] ∨ A[i − 1][j − S[i]].
                End For
        End For
        Return A[n][T].
End SubsetSum
```

- What is the runtime of this algorithm?
  - Recover is $\Theta(n)$.
  - Subset Sum is $\Theta(nT)$.

- Is this a good runtime?
  - Recall that it takes $t = \lfloor \log T \rfloor + 1$ bits to write down T.
  - So this runtime is $\Theta(n2^t)$. It is exponential in t.

Partition

- Partition Problem
  - You have a set of positive integers and you want to know if you can split the set into two sets with the same sum.
  - Example: $\{5,7,2,3,9\}$?

- Can we express this problem formally?

Partition(PART)
Input: $S[1 \dots n]$ a list of n positive integers.
Output: $A \subseteq \{1, \dots, n\}$ a set of indexes into S such that $\sum_{i \in A} S[i] = \sum_{i \notin A} S[i]$ if such an A exists and false otherwise.

- Can we design a brute force algorithm for this problem?
  - We can check every subset to see its sum. How many subsets are there?

$$2^n$$

  - How many additions does it take to check each sum in the worst case?

$$n$$

  - What is the worst case runtime of the brute force algorithm?

$$\theta(n2^n)$$

Partition and Subset Sum

- Say we already have a solution to Subset Sum (we do!) and we are looking for a solution to Partition.
- Can we use the Subset Sum solution to our advantage?

```
Partition(S[1 … n])
        Let T = (1/2) Σ_{i=1}^{n} S[i] be half the sum of all elements in S.
        Return SubsetSum(S[1 … n], T).
End Partition
```

- This solution directly uses the solution to Subset Sum as a sub-routine.
  - This type of solution is called a reduction.
  - This is like a self-reduction but between two problems instead of from a problem back to itself.
  - Reductions can be used to compare the complexity of problems (we'll see more of this later).

- We have just shown that the problem Partition reduces to the problem Subset Sum.
  - This means, at least in one sense, that Partition is no harder than Subset Sum.

- Can we show the reduction the other direction and prove that Subset Sum reduces to Partition?
  - This reduction is trickier and we'll look at it again later.
  - If we can show this, then these two problems are equal in complexity. (They are!)

TCSS 343 – Design and Analysis of Algorithms

Knapsack

Dynamic Programming

- Dynamic Programming Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Identify overlapping sub-problems.
  5. Select an appropriate data structure to store the solutions to sub-problems.
  6. Design an iterative algorithm that solves the sub-problems from easiest to hardest using the self-reduction and stores their solutions in the data structure.
  7. Analyse the runtime of the algorithm.
- Additional resources on Dynamic Programming:
  - Dynamic_programming
  - Subset_sum_problem
  - Partition_problem
  - Knapsack_problem

Knapsack

- Knapsack Problem
  - You are a career thief that stuffs valuable items in your favorite thief's knapsack. Your knapsack has a capacity of C pounds. Your next job has a collection of n valuable items each with a weight and a value. You need to decide which items to take such that they will all fit in the knapsack and you maximize the value of the items.

- Can we express this problem formally?

Knapsack(0-1 KNAP)
Input: $W[1 \dots n]$ a list of n positive weights $V[1 \dots n]$ a list of n positive values and C the capacity of the knapsack.
Output: m a maximum earnings and $A \subseteq \{1, \dots, n\}$ a set of indexes of items such that $C \geq \sum_{i \in A} W[i]$ and $m = \sum_{i \in A} V[i]$.

- Can we design a brute force algorithm for this problem?
  - We can check every set of items and maximize over the sets that fit in the knapsack. How many subsets are there?

$$2^n$$

  - How many additions does it take to check each subset in the worst case?

$$2n$$

  - What is the worst-case runtime of the brute force algorithm?

$$\theta(n2^n)$$

Knapsack

Knapsack(0-1 KNAP)
Input: $W[1 \dots n]$ a list of n positive weights $V[1 \dots n]$ a list of n positive values and C the capacity of the knapsack.
Output: m a maximum earnings and $A \subseteq \{1, \dots, n\}$ a set of indexes of items such that $C \geq \sum_{i \in A} W[i]$ and $m = \sum_{i \in A} V[i]$.

- What is a good set of sub-problems to look at?
  - We can reduce the number of items to a smaller set.
  - We can also select different capacities for our knapsack.
  - For this problem we will consider sub-problems that restrict both dimensions.

- Let $K(n, C)$ express the output for the Knapsack problem on input $W[1 \dots n]$, $V[1 \dots n]$ and C. Can we state a self-reduction from this problem to its sub-problems? We consider here only on finding the maximum earnings.

$$K(n, C) = \begin{cases} 0 & \text{if } n = 0 \text{ or } C = 0 \\ \max\{K(n-1, C), K(n-1, C - W[n]) + V[n]\} & \text{if } n > 0 \text{ and } C > 0 \end{cases}$$

- Do we have overlapping sub-problems?

Data Structure

$$K(n, C) = \begin{cases} 0 & \text{if } n = 0 \text{ or } C = 0 \\ \max\{K(n-1, C), K(n-1, C - W[n]) + V[n]\} & \text{if } n > 0 \text{ and } C > 0 \end{cases}$$

- What type of data structure would be appropriate for this problem?
  - In this problem we use two variables to indicate the sub-problem (n and C).
  - So we have a 2-dimensional problem.
  - We will create a 2d array $A[0 \ldots n][0 \ldots C]$ to store the solutions to the sub-problems .

- What order should we evaluate the problems?
  - We will fill in the array considering one item at a time in order from 0 to n.
  - For each item we will consider each target value at a time in order from 0 to c.

Dynamic Programming Algorithm

$$K(n, C) = \begin{cases} 0 & \text{if } n = 0 \text{ or } C = 0 \\ \max\{K(n-1, C), K(n-1, C - W[n]) + V[n]\} & \text{if } n > 0 \text{ and } C > 0 \end{cases}$$

```
Knapsack(V[1 ... n], W[1 ... n], C)
        Let A[0 ... n][0 ... C] be an array of integers of size n + 1 × c + 1.
        For i = 0 to n:
                Let A[i][0] = 0.
        End For
        For j = 0 to C:
                Let A[0][j] = 0.
        End For
        For i = 1 to n do:
                For j = 1 to C:
                        A[i][j] = max{A[i − 1][j], A[i − 1][j − W[i]] + V[i]}.
                End For
        End For
        Return A[n][C].
End Knapsack
```

Dynamic Programming Algorithm

```
Knapsack(V[1 ... n], W[1 ... n], C)
        Let A[0 ... n][0 ... C] be an array of integers of size n + 1 × c + 1.
        For i = 0 to n:
                Let A[i][0] = 0.
        End For
        For j = 0 to C:
                Let A[0][j] = 0.
        End For
        For i = 1 to n do:
                For j = 1 to C:
                        A[i][j] = max{A[i − 1][j], A[i − 1][j − W[i]] + V[i]}.
                End For
        End For
        Return A[n][C].
End Knapsack
```

$$W = [1,4,2,6,7,4,1,3], V = [4,1,5,6,7,2,4,1], c = 13$$

	A	0	1	2	3	4	5	6	7	8	9	10	11	12	13
		0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	4	0	4	4	4	4	4	4	4	4	4	4	4	4	4
4	1	0	4	4	4	4	5	5	5	5	5	5	5	5	5
2	5	0	4	5	9	9	9	9	10	10	10	10	10	10	10
6	6	0	4	5	9	9	9	9	10	11	15	15	15	15	16
7	7	0	4	5	9	9	9	9	10	11	15	16	16	16	16
4	2	0	4	5	9	9	9	9	11	11	15	16	16	16	17
1	4	0	4	8	9	13	13	13	13	15	15	19	20	20	20
3	1	0	4	8	9	13	13	13	14	15	15	19	20	20	20

Dynamic Programming Algorithm

```
Knapsack(V[1 ... n], W[1 ... n], C)
        Let A[0 ... n][0 ... C] be an array of integers of size n + 1 × c + 1.
        For i = 0 to n:
                Let A[i][0] = 0.
        End For
        For j = 0 to C:
                Let A[0][j] = 0.
        End For
        For i = 1 to n do:
                For j = 1 to C:
                        A[i][j] = max{A[i − 1][j], A[i − 1][j − W[i]] + V[i]}.
                End For
        End For
        Return A[n][C].
End Knapsack
```

- What is the runtime of this algorithm?
  - Recover is $\Theta(n)$.
  - Knapsack is $\Theta(nC)$.

- Is this a good runtime?
  - Recall that it takes $c = \lfloor \log C \rfloor + 1$ bits to write down C.
  - So this runtime is $\Theta(n2^c)$. It is exponential in c.